PROGRAMMING IN THE UNIX ENVIRONMENT, DV1457/DV1578

LAB 2: C PROGRAMMING

Amir Yavariabdi

Blekinge Institute of Technology

Contact: aya@bth.se **Date: September 1, 2024**

Exercise 1: Contact Management Program

Objective:

The goal of this exercise is to develop a C program that can manage contact information for up to 12 individuals. The program should enable users to perform the following tasks:

- Add a new contact
- Delete an existing contact
- Update the details of an existing contact
- Display all stored contacts

Additionally, the program must save all contact information to a text file named **Tel.txt** and ensure that the file is updated whenever changes are made.

Tasks (Functions):

To achieve the above objectives, the program is divided into the following six functions:

1. *readContactsFromFile(struct Contact contacts[], int *contactCount)*

Purpose: This function reads existing contact information from Tel.txt and initializes the contact's array. It updates the contact count accordingly.

Details: This function uses file I/O to open and read from Tel.txt. If the file is not found, the function should handle it gracefully by initializing an empty contacts array.

2. *writeContactsToFile(struct Contact contacts[], int contactCount)*

Purpose: This function writes the current list of contacts to Tel.txt, ensuring that any additions, deletions, or updates made by the user are saved.

Details: File I/O is used here to open the file in write mode, iterating through the contacts array and writing each contact's information to the file.

3. *addContact(struct Contact contacts[], int *contactCount)*

Purpose: This function allows the user to add a new contact to the list, provided the list has not already reached its maximum capacity.

Details: It prompts the user for the contact's first name, last name, and phone number, then adds this information to the contacts array and increments the contact count.

4. *deleteContact(struct Contact contacts[], int *contactCount)*

Purpose: This function removes a contact from the list based on the user's input, which includes the first and last names of the contact to be deleted.

Details: The function searches for the contact and, if found, removes it by shifting the subsequent contacts in the array to fill the gap. The contact count is then decremented.

5. *updateContact(struct Contact contacts[], int contactCount)*

Purpose: This function allows the user to modify the details of an existing contact.

Details: The user is prompted for the contact’s first and last names to locate the entry. If the contact is found, the user can input new values for the first name, last name, and phone number, which overwrite the existing data.

6. *displayContacts(struct Contact contacts[], int contactCount)*

Purpose: This function displays all current contacts to the user in a formatted manner.

Details: It iterates through the contacts array and prints each contact's details (first name, last name, and phone number) in a tabular format. If no contacts are available, it informs the user.

Code Explanation for Exercise 1

Below is a line-by-line explanation of the contact management program code, followed by an overview of important C syntaxes and strategies used.

1. Include Necessary Headers

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

- #include <stdio.h>: Includes the Standard Input Output header file, which provides functionalities for input and output operations, such as printf and scanf.
- #include <stdlib.h>: This provides functions for memory allocation, process control, and other utility functions.
- #include <string.h>: Includes the String library header, which provides functions for manipulating C strings, such as strcmp (compares two strings character by character. If the strings are equal, the function returns 0) and strcpy.

2. Define Constants

```
#define MAX_CONTACTS 12
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
```

- #define MAX_CONTACTS 12: Defines a constant for the maximum number of contacts the program can handle.
- #define MAX_NAME_LENGTH 50: Defines a constant for the maximum length of names.
- #define MAX_PHONE_LENGTH 15: Defines a constant for the maximum length of phone numbers

3. Define the Contact Structure

```
struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};
```

- struct Contact: This structure represents a single contact, containing the first name, last name, and phone number, each of which is an array of characters

4. Read Contacts from File:

```
void readContactsFromFile(struct Contact contacts[], int *contactCount) {
    FILE *file = fopen("Tel.txt", "r");
    if (file == NULL) {
        printf("No existing contacts found.\n");
        return;
    }
    *contactCount = 0;
    while (fscanf(file, "%s %s %s", contacts[*contactCount].firstName,
                  contacts[*contactCount].lastName,
                  contacts[*contactCount].phoneNumber) != EOF) {
        (*contactCount)++;
    }
    fclose(file);
}
```

- FILE *file = fopen("Tel.txt", "r");: Opens the Tel.txt file in read mode.
- if (file == NULL): Checks if the file exists. If not, it prints a message and returns.
- while with fscanf(...): Reads each contact's details from the file into the contacts array until the end of the file (EOF) is reached.
- (*contactCount)++: Increments the contact count for each contact read.

  Purpose of Using a Pointer for contactCount: In C, function parameters are passed by value by default, meaning a copy of the parameter is made when passed to the function. If contactCount were passed directly as an integer (not a pointer), any changes made to contactCount inside the function would only affect the copy, not the original variable in the calling function (e.g., in main). By passing contactCount as a pointer (int *contactCount), the function operates directly on the memory address of the original variable. This means that any changes to *contactCount (the value at the memory address) inside the function will directly update the original contactCount variable in the calling function. Then, without using a pointer, the function would be unable to modify the original contactCount variable, and the calling function would not know how many contacts were read.
- fclose(file);: Closes the file after reading.

5. Write Contacts to File

```
void writeContactsToFile(struct Contact contacts[], int contactCount) {
    FILE *file = fopen("Tel.txt", "w");
    if (file == NULL) {
        printf("Error opening file for writing.\n");
        return;
    }

    for (int i = 0; i < contactCount; i++) {
        fprintf(file, "%s %s %s\n", contacts[i].firstName, contacts[i].lastName,
contacts[i].phoneNumber);
    }

    fclose(file);
}
```

- FILE *file = fopen("Tel.txt", "w");: Opens the Tel.txt file in write mode. If the file doesn't exist, it creates it.

- for (int i = 0; i < contactCount; i++): Iterates through each contact and writes its details to the file.
- fprintf(...): Formats the contact's details into a string and writes it to the file.
- fclose(file);: Closes the file after writing.

6. Add a New Contact

```
void addContact(struct Contact contacts[], int *contactCount) {
    if (*contactCount >= MAX_CONTACTS) {
        printf("Contact list is full. Cannot add more contacts.\n");
        return;
    }
    printf("Enter first name: ");
    scanf("%s", contacts[*contactCount].firstName);
    printf("Enter last name: ");
    scanf("%s", contacts[*contactCount].lastName);
    printf("Enter phone number: ");
    scanf("%s", contacts[*contactCount].phoneNumber);
    (*contactCount)++;
    printf("Contact added successfully.\n");
}
```

- if (*contactCount >= MAX_CONTACTS): Checks if the contact list is full.
- scanf("%s", contacts[*contactCount].firstName);: Prompts the user to input the first name, last name, and phone number, storing them in the respective fields of the contacts array.
- (*contactCount)++: Increments the contact count after adding the new contact.
- printf("Contact added successfully.\n");: Confirms that the contact was added.

7. Delete a Contact

```
void deleteContact(struct Contact contacts[], int *contactCount) {
    char firstName[MAX_NAME_LENGTH], lastName[MAX_NAME_LENGTH];
    printf("Enter first name of the contact to delete: ");
    scanf("%s", firstName);
    printf("Enter last name of the contact to delete: ");
    scanf("%s", lastName);

    int found = 0;
    for (int i = 0; i < *contactCount; i++) {
        if (strcmp(contacts[i].firstName, firstName) == 0 && strcmp(contacts[i].lastName, lastName) == 0) {
            for (int j = i; j < *contactCount - 1; j++) {
                contacts[j] = contacts[j + 1];
            }
            (*contactCount)--;
            found = 1;
            printf("Contact deleted successfully.\n");
            break;
        }
    }

    if (!found) {
        printf("Contact not found.\n");
    }
}
```

- char firstName[MAX_NAME_LENGTH], lastName[MAX_NAME_LENGTH];: Variables to store the first and last names of the contact to delete.
- for (int i = 0; i < *contactCount; i++): Iterates through the contacts to find a match.
- if (strcmp(contacts[i].firstName, firstName) == 0 && strcmp(contacts[i].lastName, lastName) == 0): Compares the names to find the correct contact to delete.
- for (int j = i; j < *contactCount - 1; j++): Shifts remaining contacts to fill the gap left by the deleted contact.
- (*contactCount)--;: Decreases the contact count after deletion.
- printf("Contact deleted successfully.\n");: Confirms that the contact was deleted.

8. Update a Contact

```
void updateContact(struct Contact contacts[], int contactCount) {
    char firstName[MAX_NAME_LENGTH], lastName[MAX_NAME_LENGTH];
    printf("Enter first name of the contact to update: ");
    scanf("%s", firstName);
    printf("Enter last name of the contact to update: ");
    scanf("%s", lastName);

    int found = 0;
    for (int i = 0; i < contactCount; i++) {
        if (strcmp(contacts[i].firstName, firstName) == 0 && strcmp(contacts[i].lastName, lastName) == 0) {
            printf("Enter new first name: ");
            scanf("%s", contacts[i].firstName);
            printf("Enter new last name: ");
            scanf("%s", contacts[i].lastName);
            printf("Enter new phone number: ");
            scanf("%s", contacts[i].phoneNumber);
            found = 1;
            printf("Contact updated successfully.\n");
            break;
        }
    }

    if (!found) {
        printf("Contact not found.\n");
    }
}
```

- Prompts the user for the first and last names of the contact they wish to update.
- Searches for the contact in the array using strcmp. The strcmp function in C is used to compare two strings. It is part of the C Standard Library and is declared in the <string.h> header file.
- If contact found, it asks the user to enter new values for the first name, last name, and phone number, and updates the contact.
- If contact not found, it notifies the user.

9. Display All Contacts

```
void displayContacts(struct Contact contacts[], int contactCount) {
    if (contactCount == 0) {
        printf("No contacts to display.\n");
        return;
    }

    printf("\n%-20s %-20s %-15s\n", "First Name", "Last Name", "Phone Number");
    printf("-------------------------------------------------------------\n");
    for (int i = 0; i < contactCount; i++) {
        printf("%-20s %-20s %-15s\n", contacts[i].firstName, contacts[i].lastName,
contacts[i].phoneNumber);
    }
}
```

- Checks if there are any contacts to display.
- Prints out the contacts in a formatted table with columns for first name, last name, and phone number.

10. Main Function

```
int main() {
    struct Contact contacts[MAX_CONTACTS];  // Array to store contact information
    int contactCount = 0;  // Number of contacts currently stored
    readContactsFromFile(contacts, &contactCount);  // Load existing contacts from file
    int choice = 0;  // Variable to store user's menu choice
    while (choice != 5) {  // Continue looping until the user chooses to exit (option 5)
        // Display menu options
        printf("\nContact Management System\n");
        printf("1. Add Contact\n");
        printf("2. Delete Contact\n");
        printf("3. Update Contact\n");
        printf("4. Display Contacts\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);  // Get the user's choice
        // Perform the corresponding action based on the user's choice
        if (choice == 1) {
            addContact(contacts, &contactCount);  // Add a new contact
        } else if (choice == 2) {
            deleteContact(contacts, &contactCount);  // Delete an existing contact
        } else if (choice == 3) {
            updateContact(contacts, contactCount);  // Update contact information
        } else if (choice == 4) {
            displayContacts(contacts, contactCount);  // Display all contacts
        } else if (choice != 5) {  // If the choice is not 1-5, inform the user
            printf("Invalid choice, please try again.\n");
        }
    }
    writeContactsToFile(contacts, contactCount);  // Save contacts to file before exiting
    return 0;  // Exit the program
}
```

- *Array Initialization:*
  struct Contact contacts[MAX_CONTACTS];: This initializes an array of Contact structures that will hold the information of up to MAX_CONTACTS people.

  int contactCount = 0;: This variable keeps track of how many contacts are currently stored in the array.

- *Loading Existing Contacts:*
  readContactsFromFile(contacts, &contactCount);: This function reads the existing contacts from the Tel.txt file and fills the contacts array and contactCount.

- *Menu Loop:*
  int choice = 0;: The choice variable is initialized to store the user's input from the menu.

  while (choice != 5) { ... }: This while loop runs repeatedly until the user selects option 5, which exits the program. Inside the loop, the program displays a menu and prompts the user for their choice.

- *Handling User Input:*
  if (choice == 1) { ... } else if (choice == 2) { ... } else if (choice == 3) { ... } else if (choice == 4) { ... }: These conditional statements check the user's input and call the corresponding function (addContact, deleteContact, updateContact, or displayContacts) based on the selected option.

  else if (choice != 5): If the user enters a number that is not between 1 and 5, they are prompted that their choice is invalid.

- *Saving Contacts:*
  writeContactsToFile(contacts, contactCount);: After the loop ends (when the user chooses to exit), this function saves the current state of the contacts array back to the Tel.txt file.

- *Exiting the Program:*
  return 0;: This statement terminates the program successfully.

C Syntax and Strategies:

1. *Arrays and Structs:*

- Arrays: The program uses arrays to store multiple contacts, making it easy to manage and access individual contacts through their index in the array.
- Structs: The struct Contact is used to summarize contact details, grouping related information (first name, last name, phone number) into a single entity. This structure helps in organizing data and makes the code more readable and maintainable.

2. *Pointers:*

- Use in Function Arguments: Pointers are crucial for passing the contact count to functions like addContact and deleteContact. Using pointers allows these functions to modify the contact count directly, reflecting the changes across the program.
- Pass-by-Reference: The concept of pass-by-reference using pointers ensures that updates to the contact count or modifications within the contacts array are maintained throughout the program.

3. *File I/O:*

- File Operations: The program uses standard file handling functions (fopen, fclose, fprintf, fscanf) to read from and write to Tel.txt. This is essential for saving the contact information between different runs of the program.
- Error Handling: The functions that deal with file operations include basic error handling to check if a file was successfully opened before attempting to read or write.

4. *Conditional Statements:*

- Decision Making: The program employs if-else statements to handle different scenarios, such as whether a contact was found for deletion or if the contact list is full. This ensures that the program behaves as expected in various situations.
- Control Flow: Conditional statements direct the program flow based on user input, ensuring the correct function is executed.

5. *Looping Constructs:*

- for Loop: Used to iterate through the contacts array when displaying contacts, deleting a contact, or updating a contact. This is necessary for performing actions on all or a specific subset of contacts.
- while Loop: The menu-driven interface is implemented using a while loop, which ensures that the user is presented with the menu at least once and can continue making choices until they decide to exit.

6. *String Handling:*

- String Functions: The program uses functions like strcmp to compare strings, which is essential for operations like searching for a contact by name. Handling strings correctly is critical for accurate contact management.
- String Arrays: Contact names and phone numbers are stored in character arrays, requiring careful management to avoid overflow and ensure correct string operations.

7. *Dynamic Management:*

- Static vs. Dynamic Allocation: Although the program statically allocates an array for up to 12 contacts, the structure of the program can be adapted for dynamic memory allocation if the contact list size needs to be flexible. **This could be a topic for further exploration.**

Exercise 2: Modular Contact Management Program with Object Files and Makefile

Objective:

The goal of this exercise is to modularize the contact management program from Exercise 1 by separating each function into its own C file, generating object files, and using a Makefile to compile and link these files into a single executable. This approach promotes better code organization, easier debugging, and efficient compilation.

Tasks (Functions):

Similar to Exercise 1, but now each function will reside in its own C file, and we will compile them into object files and then link them to create the final executable.

Code Explanation for Exercise 2

1. Define the Structure and Functions

Since we are not using a separate header file, we will define the Contact structure and the function prototypes in the **main.c** file.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_CONTACTS 12
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};
// Function prototypes
void readContactsFromFile(struct Contact contacts[], int *contactCount);
void writeContactsToFile(struct Contact contacts[], int contactCount);
void addContact(struct Contact contacts[], int *contactCount);
void deleteContact(struct Contact contacts[], int *contactCount);
void updateContact(struct Contact contacts[], int contactCount);
void displayContacts(struct Contact contacts[], int contactCount);
int main() {
    struct Contact contacts[MAX_CONTACTS];
    int contactCount = 0;
    readContactsFromFile(contacts, &contactCount);
    int choice = 0;  // Variable to store user's menu choice
    while (choice != 5) {  // Continue looping until the user chooses to exit (option 5)
        // Display menu options
        printf("\nContact Management System\n");
        printf("1. Add Contact\n");
        printf("2. Delete Contact\n");
        printf("3. Update Contact\n");
        printf("4. Display Contacts\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);  // Get the user's choice
        // Perform the corresponding action based on the user's choice
        if (choice == 1) {
            addContact(contacts, &contactCount);  // Add a new contact
        } else if (choice == 2) {
            deleteContact(contacts, &contactCount);  // Delete an existing contact
        } else if (choice == 3) {
            updateContact(contacts, contactCount);  // Update contact information
        } else if (choice == 4) {
            displayContacts(contacts, contactCount);  // Display all contacts
        } else if (choice != 5) {  // If the choice is not 1-5, inform the user
            printf("Invalid choice, please try again.\n");
        }
    }
    writeContactsToFile(contacts, contactCount);  // Save contacts to file before exiting
    return 0;
}
```

2. Separate Each Function into Its Own C File

Now, each function will be placed in its own C file. For example, the readContactsFromFile function will be in readContacts.c, and so on.

- **File: readContacts.c**

```
#include <stdio.h>
#include <string.h>
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
#define MAX_CONTACTS 100

struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};
void readContactsFromFile(struct Contact contacts[], int *contactCount) {
    FILE *file = fopen("Tel.txt", "r");
    if (file == NULL) {
        printf("No existing contacts found.\n");
        return;
    }
    *contactCount = 0;
    while (fscanf(file, "%s %s %s", contacts[*contactCount].firstName,
                  contacts[*contactCount].lastName,
                  contacts[*contactCount].phoneNumber) != EOF) {
        (*contactCount)++;
    }
    fclose(file);
    }
```

- **File: writeContacts.c**

```
#include <stdio.h>
#include <string.h>
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
#define MAX_CONTACTS 100
struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};
void writeContactsToFile(struct Contact contacts[], int contactCount) {
    FILE *file = fopen("Tel.txt", "w");
    if (file == NULL) {
        printf("Error opening file for writing.\n");
        return;
    }
    for (int i = 0; i < contactCount; i++) {
        fprintf(file, "%s %s %s\n", contacts[i].firstName, contacts[i].lastName,
contacts[i].phoneNumber);
    }
    fclose(file);
}
```

- **File: addContact.c**

```
#include <stdio.h>
#include <string.h>
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
#define MAX_CONTACTS 100

struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};
void addContact(struct Contact contacts[], int *contactCount) {
    if (*contactCount >= MAX_CONTACTS) {
        printf("Contact list is full. Cannot add more contacts.\n");
        return;
    }
    printf("Enter first name: ");
    scanf("%s", contacts[*contactCount].firstName);
    printf("Enter last name: ");
    scanf("%s", contacts[*contactCount].lastName);
    printf("Enter phone number: ");
    scanf("%s", contacts[*contactCount].phoneNumber);
    (*contactCount)++;
    printf("Contact added successfully.\n");
}
```

- **File: deleteContact.c**

```
#include <stdio.h>
#include <string.h>
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
#define MAX_CONTACTS 100

struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};

void deleteContact(struct Contact contacts[], int *contactCount) {
    char firstName[MAX_NAME_LENGTH], lastName[MAX_NAME_LENGTH];
    printf("Enter first name of the contact to delete: ");
    scanf("%s", firstName);
    printf("Enter last name of the contact to delete: ");
    scanf("%s", lastName);

    int found = 0;
    for (int i = 0; i < *contactCount; i++) {
        if (strcmp(contacts[i].firstName, firstName) == 0 && strcmp(contacts[i].lastName, lastName) == 0) {
            for (int j = i; j < *contactCount - 1; j++) {
                contacts[j] = contacts[j + 1];
            }
            (*contactCount)--;
```

```
            found = 1;
            printf("Contact deleted successfully.\n");
            break;
        }
    }

    if (!found) {
        printf("Contact not found.\n");
    }
}
```

- **File: updateContact.c**

```
#include <stdio.h>
#include <string.h>
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
#define MAX_CONTACTS 100

struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};

void updateContact(struct Contact contacts[], int contactCount) {
    char firstName[MAX_NAME_LENGTH], lastName[MAX_NAME_LENGTH];
    printf("Enter first name of the contact to update: ");
    scanf("%s", firstName);
    printf("Enter last name of the contact to update: ");
    scanf("%s", lastName);

    int found = 0;
    for (int i = 0; i < contactCount; i++) {
        if (strcmp(contacts[i].firstName, firstName) == 0 && strcmp(contacts[i].lastName, lastName) == 0) {
            printf("Enter new first name: ");
            scanf("%s", contacts[i].firstName);
            printf("Enter new last name: ");
            scanf("%s", contacts[i].lastName);
            printf("Enter new phone number: ");
            scanf("%s", contacts[i].phoneNumber);
            found = 1;
            printf("Contact updated successfully.\n");
            break;
        }
    }

    if (!found) {
        printf("Contact not found.\n");
    }
}
```

- **File: displayContacts.c**

```
#include <stdio.h>
#include <string.h>
#define MAX_NAME_LENGTH 50
#define MAX_PHONE_LENGTH 15
#define MAX_CONTACTS 100

struct Contact {
    char firstName[MAX_NAME_LENGTH];
    char lastName[MAX_NAME_LENGTH];
    char phoneNumber[MAX_PHONE_LENGTH];
};

void displayContacts(struct Contact contacts[], int contactCount) {
    if (contactCount == 0) {
        printf("No contacts to display.\n");
        return;
    }

    printf("\n%-20s %-20s %-15s\n", "First Name", "Last Name", "Phone Number");
    printf("-------------------------------------------------------\n");
    for (int i = 0; i < contactCount; i++) {
        printf("%-20s %-20s %-15s\n", contacts[i].firstName, contacts[i].lastName,
contacts[i].phoneNumber);
    }
}
```

3. Create a Makefile

A Makefile simplifies the build process by automating the compilation and linking of multiple source files into a single executable. Open the terminal and run: **gedit Makefile &**

```
# Target: contact_manager
contact_manager: main.o readContacts.o writeContacts.o addContact.o deleteContact.o
updateContact.o displayContacts.o
	gcc -o contact_manager main.o readContacts.o writeContacts.o addContact.o
deleteContact.o updateContact.o displayContacts.o

main.o: main.c
	gcc -c main.c
readContacts.o: readContacts.c
	gcc -c readContacts.c

writeContacts.o: writeContacts.c
	gcc -c writeContacts.c
addContact.o: addContact.c
	gcc -c addContact.c
deleteContact.o: deleteContact.c
	gcc -c deleteContact.c
updateContact.o: updateContact.c
	gcc -c updateContact.c
displayContacts.o: displayContacts.c
	gcc -c displayContacts.c
```

4. Compile and Run the Program

To compile and run the program:

- Open the terminal.
- Navigate to the directory containing your .c files and Makefile.
- Run **make** to compile the program.
- Run the executable: **./contact_manager**

C Syntax and Strategies:

1. *Object Files and Linking*

- Object Files: Each .c file is compiled into an object file (.o), which contains machine code but not linked together yet.
- Linking: The linker combines these object files into a single executable, resolving references between different parts of the code.

2. *Makefile*

- Automation: The Makefile automates the compilation and linking process, ensuring that all changes are reflected in the executable.
- Simplification: Simplifies the build process, especially for larger projects with multiple source files.

3. *Header*

- Header Files: For more advanced organization, consider using header files (.h) to define structures and function prototypes, reducing repetition and improving code modularity.

Exercise 3: Calculating Euclidean Distance Between Points

Objective:

- Data: Preparing the example data file
- Read Data from File: Load a 2D matrix from a file.
- Calculate Euclidean Distance: Compute distances between matrix rows.
- Save Distance Matrix: Write the distance matrix to a file.

Tasks:

In this exercise, we will build a C program that reads data points from a file, calculates the Euclidean distance between each pair of points, and writes the resulting distance matrix to a new file.

- Data Input: We have a set of points, each described by multiple features. These points will be read from a file.
- Distance Calculation: We will calculate the Euclidean distance between every pair of points. The Euclidean distance formula is used to find the distance between two points on a plane. This formula says the distance between two points, for instance (x_1, y_1) and (x_2, y_2) is $d = \sqrt{[x_2 - x_1]^2 + [y_2 - y_1]^2}$.
- Data Output: The results (distance matrix) will be written to a new file.

Code Explanation for Exercise 3

1. Preparing the Example Data File

First, we need a file containing some data points. Let's create a file named **data.txt** with the following content:

```
3 2
1.0 2.0
3.0 4.0
5.0 6.0
```

- This file represents a 3 × 2 matrix, where each row is a point in 2-dimensional space.
- The actual data points: (1.0, 2.0), (3.0, 4.0), and (5.0, 6.0).

2. Reading the Matrix from a File

The first block of our program is responsible for reading the matrix from a file and storing it in memory.

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

double** readMatrixFromFile(const char* filename, int* rows, int* cols) {
    FILE* file = fopen(filename, "r");
    if (file == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE); // Exit if file opening fails
    }
    // Read the number of rows and columns
    fscanf(file, "%d %d", rows, cols);

    // Allocate memory for the matrix (array of pointers to arrays)
    double** matrix = (double**)malloc(*rows * sizeof(double*));
    for (int i = 0; i < *rows; i++) {
        matrix[i] = (double*)malloc(*cols * sizeof(double));

        // Read each element in the row
        for (int j = 0; j < *cols; j++) {
            fscanf(file, "%lf", &matrix[i][j]);
        }
    }
    fclose(file);
    return matrix; // Return the pointer to the matrix
}
```

- #include <stdlib.h>: Includes the standard library for memory allocation and other utilities.
- FILE* file = fopen(filename, "r");: Opens the specified file in read mode.
- fscanf(file, "%d %d", rows, cols);: Reads the number of rows and columns from the file.
- double** matrix = (double**)malloc(*rows * sizeof(double*));: Allocates memory for an array of pointers (rows).
- matrix[i] = (double*)malloc(*cols * sizeof(double));: Allocates memory for each row's data (columns).
- fscanf(file, "%lf", &matrix[i][j]);: Reads the matrix elements from the file into the allocated memory.

3. Calculating the Euclidean Distance

```
double euclideanDistance(double* point1, double* point2, int size) {
    double sum = 0.0;
    for (int i = 0; i < size; i++) {
        double diff = point1[i] - point2[i];
        sum += diff * diff;
    }
    return sqrt(sum);
}
```

- euclideanDistance: This function calculates the Euclidean distance between two points.
- double sum = 0.0;: Initializes the sum of squared differences.
- double diff = point1[i] - point2[i];: Calculates the difference between the corresponding dimensions of the two points.
- sum += diff * diff;: Squares the difference and adds it to the sum.
- return sqrt(sum);: Returns the square root of the sum, which is the Euclidean distance.

4. Calculating the Distance Matrix

```
double** calculateDistanceMatrix(double** matrix, int rows, int cols) {
    double** distanceMatrix = (double**)malloc(rows * sizeof(double*));
    for (int i = 0; i < rows; i++) {
        distanceMatrix[i] = (double*)malloc(rows * sizeof(double));
        for (int j = 0; j < rows; j++) {
            if (i == j) {
                distanceMatrix[i][j] = 0.0; // Distance from a point to itself is zero
            } else {
                distanceMatrix[i][j] = euclideanDistance(matrix[i], matrix[j], cols);
            }
        }
    }
    return distanceMatrix;
}
```

- double** distanceMatrix = (double**)malloc(rows * sizeof(double*));: Allocates memory for the distance matrix.
- for (int i = 0; i < rows; i++): Iterates over each row of the matrix.
- distanceMatrix[i] = (double*)malloc(rows * sizeof(double));: Allocates memory for each row in the distance matrix.
- for (int j = 0; j < rows; j++): Iterates over each column (other point) for distance calculation.
- if (i == j): Checks if comparing a point with itself.
- distanceMatrix[i][j] = 0.0;: Sets the distance from a point to itself as zero.
- else: If comparing two different points.
- distanceMatrix[i][j] = euclideanDistance(matrix[i], matrix[j], cols);: Calls the euclideanDistance function to compute the distance.

5. Writing the Distance Matrix to a File

```
void writeMatrixToFile(const char* filename, double** matrix, int rows, int cols) {
    FILE* file = fopen(filename, "w");
    if (file == NULL) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            fprintf(file, "%lf ", matrix[i][j]);
        }
        fprintf(file, "\n");
    }

    fclose(file);
}
```

- FILE* file = fopen(filename, "w");: Opens the specified file in write mode.
- if (file == NULL): Checks if the file was successfully opened. If not, an error message is printed, and the program exits.
- fprintf(file, "%lf ", matrix[i][j]);: Writes the matrix element to the file.
- fprintf(file, "\n");: Moves to the next line after writing each row.
- fclose(file);: Closes the file after writing.

6. Freeing Allocated Memory

```
void freeMatrix(double** matrix, int rows) {
    for (int i = 0; i < rows; i++) {
        free(matrix[i]);
    }
    free(matrix);
}
```

- for (int i = 0; i < rows; i++): Iterates over each row of the matrix.
- free(matrix[i]);: Frees the memory allocated for each row.
- free(matrix);: Frees the memory allocated for the array of row pointers.

7. Main Function

```
int main() {
    int rows, cols;
    double** matrix = readMatrixFromFile("data.txt", &rows, &cols);

    // Calculate the distance matrix
    double** distanceMatrix = calculateDistanceMatrix(matrix, rows, cols);

    // Write the distance matrix to a file
    writeMatrixToFile("distances.txt", distanceMatrix, rows, rows);

    // Free allocated memory
    freeMatrix(matrix, rows);
    freeMatrix(distanceMatrix, rows);
    return 0;
}
```

- int rows, cols;: Declares variables to store the number of rows and columns in the matrix.
- double** matrix = readMatrixFromFile("data.txt", &rows, &cols);:
  - Calls readMatrixFromFile to load the matrix from data.txt.
  - Stores the matrix dimensions in rows and cols.
  - The matrix is stored as a dynamically allocated 2D array in the matrix pointer.
- double** distanceMatrix = calculateDistanceMatrix(matrix, rows, cols);: Calculates the Euclidean distance between each pair of rows (points) in the matrix, storing the results in the distanceMatrix.
- writeMatrixToFile("distances.txt", distanceMatrix, rows, rows);: Writes the distance matrix to the file distances.txt. The distance matrix is a square matrix with dimensions equal to the number of rows in the original matrix.
- freeMatrix(matrix, rows);: Frees the memory allocated for the original matrix.
- freeMatrix(distanceMatrix, rows);: Frees the memory allocated for the distance matrix.

8. Output

The results in **distances.txt** show the Euclidean distances between the points (rows) in the original matrix. The diagonal elements are zero because the distance from a point to itself is always zero.

```
0.000000 2.828427 5.656854
2.828427 0.000000 2.828427
5.656854 2.828427 0.000000
```

Syntax and Strategies

1. *Dynamic Memory Allocation and Double Pointers:*

- Dynamic Memory Allocation: In C, memory can be dynamically allocated at runtime using functions like malloc. This is essential when the size of an array or matrix isn't known until runtime.
- Double Pointers (double**): Used to represent a dynamically allocated 2D array (a matrix). Understanding how to work with double pointers is crucial for managing multidimensional data.

2. *File I/O:*

- Reading and Writing Files: The program demonstrates how to read data from a file and write output to another file using standard C file I/O functions (fopen, fscanf, fprintf, fclose).
- Handling File Errors: It’s important to check if a file was opened successfully to avoid crashes or unexpected behavior.

3. *Euclidean Distance Calculation:*

- Mathematical Operations: The exercise involves calculating the Euclidean distance between points, a key operation in many data analysis and machine learning algorithms like K-means clustering.

4. *Memory Management:*

- Freeing Allocated Memory: Dynamic memory allocation must be paired with proper memory deallocation using free to prevent memory leaks.

5. *Functions and Modularity:*

- Modular Code: The program is divided into functions that perform specific tasks. This makes the code more organized, easier to understand, and reusable.